Project 2 – FederHub

TL;DR

FederHub is a cloud platform that lets multiple organizations collaboratively train and evaluate ML models without sharing raw data, by orchestrating federated learning rounds, monitoring client behavior, and visualizing performance.

Project Summary (table row)

Project Name	Short Description	Team Enthusiasm Rank	Team Confidence	Product Owner
FederHub	Platform to manage, monitor, and visualize federated learning across many clients without centralizing data.	7	8	Nikhil Mahana

Project Details

Name

FederHub – Privacy-Preserving Federated Learning & Analytics Platform

Description

FederHub is a platform for managing federated learning experiments across many clients that cannot share their raw data. Each client (e.g., hospital, branch, or company) trains a local copy of a model on its own dataset and periodically sends only model updates (weights or gradients) to a central aggregator. The aggregator combines these updates into a new global model and sends it back, so every participant benefits from patterns in other clients' data without exposing their own records.

Through a web interface, administrators can create federated training jobs, register clients, define training rounds, and track metrics such as global accuracy, client-level performance, participation, and fairness across clients. The system also detects issues like misbehaving clients, slow nodes, or performance drift.

Problems we want to solve:

- Organizations often cannot legally or ethically centralize data (e.g., healthcare, finance), limiting model performance.
- Existing federated learning frameworks can be complex and code-heavy, making experimentation and monitoring difficult.
- There is limited tooling to easily visualize and manage federated training runs as a productized platform.

User Personas and Major Features

User Personas

- Platform Admin
  - Manages the FederHub instance, organizations, and high-level security settings.
- Federation Owner / ML Engineer
  - Sets up federated training jobs, configures models and hyperparameters, oversees experiments.
- Client Operator (per organization)
  - Manages their local client node, connects their data, and monitors local performance.

Major Features

- Experiment management: create/edit federated training jobs, configure rounds, and model/hyperparameter templates.

- Client registration & management: securely register clients, manage tokens/keys, view status.
- Orchestration engine for training rounds (global aggregation, round scheduling, client participation tracking).
- Metrics dashboard: global vs client-specific metrics over rounds, data drift/performance trends, fairness indicators.
- Logging and alerts for failed/slow clients and anomalous updates.

MVP (Minimum Viable Product)

Must-have features:

- Basic auth and roles (platform admin, ML engineer).
- Ability to define a simple federated training job (e.g., a classification model) with parameters like number of rounds and local epochs.
- Simulated clients (containers/services) that:
    - Load a local dataset.
    - Train locally and send model updates each round.
- Aggregator service that averages client updates (e.g., FedAvg) into a new global model.
- Web UI showing:
    - Current global accuracy/loss over rounds.
    - Which clients participated in each round.
- Basic failure handling (a round can proceed if some clients drop out).

MLP (Minimum Loveable Product)

Features that make users love the platform:

- Support for real client separation (e.g., each client with its own API key, separate simulated datasets).

- Rich dashboards with:
    - Per-client metrics (accuracy, contribution, data size).
    - Fairness metrics and comparisons between clients.
- Detection of anomalous client behavior (very large or outlier updates) with warnings.
- Model registry tracking different global models and experiments over time with tags and notes.

Nice-to-have Features

- Pluggable aggregation algorithms (FedAvg, FedProx, etc.).
- Simple privacy controls/visualizations (e.g., simulate differential privacy noise).
- Support for different task types (e.g., image vs text classification).
- Export of experiment data for external analysis (CSV/JSON).

Call Out CRUD Features

Platform Admin

- C: Create organizations, user accounts, and global configuration.
- R: Read platform-wide metrics (all federations, all clients).
- U: Update organization permissions, global security policies.
- D: Deactivate organizations/users.

Federation Owner / ML Engineer

- C: Create federated training jobs, experiment configs, and model templates.
- R: Read dashboards (metrics, logs, client status, model registry).
- U: Update experiment settings (round count, aggregation, learning rates).
- D: Archive old experiments or delete test federations.

Client Operator

- C: Register a client node with the platform, connect local dataset configuration.
- R: Read local metrics, logs, and how their client performs compared to global.
- U: Update local node settings (e.g., training batch size, participation).
- D: Deregister the client from a federation.

Similar Already Existing

At least 3 similar applications/frameworks:

- **TensorFlow Federated (TFF)** – Open-source framework for building federated learning algorithms, mainly code-level tools rather than a full product with multi-user dashboards.
- **OpenFL (Intel)** – Open-source federated learning library with focus on security and flexibility; oriented toward engineers and researchers.
- **Flower / FedML** – Frameworks for building federated learning systems; they provide core orchestration but usually require more engineering effort to build UIs and experiment management layers.

FederHub aims to provide a clearer, more approachable “platform” experience on top of these ideas, emphasizing orchestration, monitoring, and usability rather than novel algorithms.